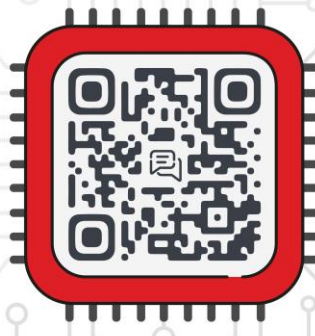




6

38

5,700 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة

Lossless Compression

د. مدحت الصوص



RB Informatcs; 04/05/2026

نظم الوسائط المتعددة

في المحاضرات السابقة، تم التعرف على طبيعة الوسائط المتعددة وكيفية تمثيل الإشارات الصوتية والصورة رقمياً من خلال عمليتي التقطيع (Sampling) والتكميم (Quantization). أما في هذه المحاضرة، فسننتقل من مرحلة التمثيل إلى مرحلة التعامل مع ضخامة هذه البيانات. سنبدأ رحلتنا في عالم الضغط غير الفاقد للبيانات (Lossless Compression)، لنفهم كيف تتيح لنا نظرية المعلومات والمفاهيم الإحصائية مثل الإنتروبيا (Entropy) تقليل حجم الملفات وتوفير موارد التخزين، مع ضمان استرجاع المعلومة الأصلية بدقة كاملة دون فقدان بت واحد منها.

الضغط بدون فقدان - Lossless Compression

هو تقنية تتيح لنا تخزين المعلومات ونقلها بكفاءة دون فقدان أي من تفاصيلها الأصلية.

ميزاته

1. حفظ الكميات الهائلة من البيانات بطريقة سهلة الاستخدام وموفرة للموارد.
2. تقليل حجم البيانات الأصلية دون التأثير على جودتها.
3. ضغط البيانات واسترجاعها دون فقدان بت واحد منها.
4. تخزين الملفات على وسيط تخزين ثم نقلها سواء كان (هارد أو فلاشة)

يوجد لدي نوع آخر من الضغط وهو الضغط مع فقدان lossy ويستخدم هذا النوع في ضغط الصوت أو الصورة (يمكن أن نخسر جزء من الداتا مقابل الحصول على صورة أو صوت واضح).

أولاً: سنتعرف كيف سنقوم بعملية الضغط ويجب أن نمتلك مقياساً يحدده لنا حيث كفاءة هذا الضغط ويحدد لنا مدى فعالية الضغط في تقليل حجم البيانات الأصلية ويقاس بالمعادلة:

$$\text{Compression Ratio} = 1 - \left(\frac{B_1}{B_0}\right)$$

حيث:

B_0 : عدد البتات الموجودة في البيانات قبل ضغطها.

B_1 : عدد البتات الموجودة في البيانات بعد ضغطها.

■ إذا كان الضغط فعال فإن البيانات الناتجة ستكون أصغر من البيانات الأصلية.
أي أن: $B_1 < B_0$

مثال:

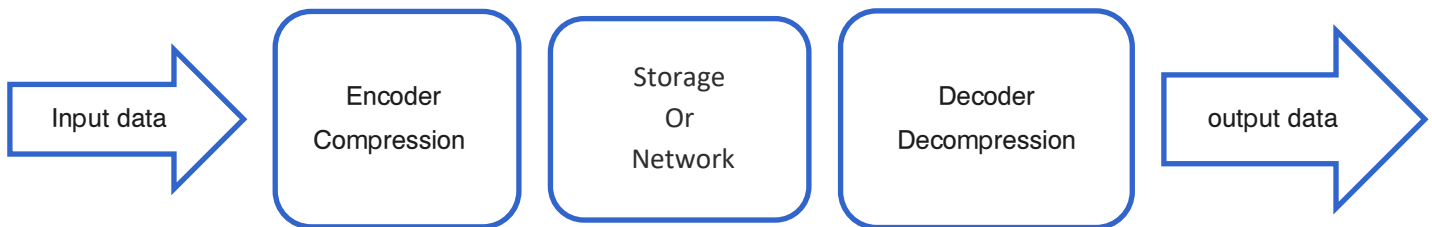
إذا كان لدي ملف حجمه 100 MB وبعد الضغط أصبح حجمه 25MB

$$\text{Compression Ratio} = 1 - \left(\frac{B_1}{B_0}\right)$$

$$\text{Compression Ratio} = 1 - \left(\frac{25}{100}\right) = \frac{3}{4}$$

أي تم ضغط الملف بنسبة 75٪.

ملاحظة: إذا كانت Compression ratio = 0 هذا يعني عدم وجود ضغط حيث $B_1 = B_0$



يتم الضغط (عن طريق الـ encoder) من أجل تخزين المعلومات أو نقلها أو إرسالها وعندما نريد معالجة هذه المعلومات مرة أخرى يجب فك ضغطها (عن طريق الـ decoder)

يوجد لدينا مجموعة من خوارزميات الضغط:

1. تقنيات إحصائية (رياضية) Statistical techniques :

تعتمد على قيم إحصائية (رياضية) أي تعتمد على تكرار ظهور الرموز والحروف داخل البيانات التي نريد أن نضغطها.

- RLE (Run Length Encoding)
- Shannon-Fano Coding
- Huffman coding
- Adaptive Huffman Coding
- Arithmetic Coding

2. تقنيات تعتمد على القاموس Dictionary techniques :

تعتمد على القواميس فهي تقوم على أساس بناء قاموس يحتوي على الكتل المتكررة والعبارات داخل البيانات.

■ LZ77

■ LZW

كل نوع له ميزاته حيث الخوارزميات الإحصائية تكون أكثر فعالية عندما يكون لدينا تكرار في البيانات بينما خوارزميات القواميس أكثر فعالية عند وجود العبارات المتكررة بشكل كبير.

سنقوم بشرح الخوارزميات واحدة تلو الأخرى.

سنبداً بالخوارزميات الإحصائية Statistical techniques of compression:

RLE

- تطبق هذه الخوارزمية على التسلسلات التي تحتوي على القيمة نفسها مكررة لعدة مرات متتالية حيث نقوم بترميز التسلسل لتخزين قيمة واحدة فقط مع عدد مرات تكرارها.
- مثال توضيحي: إذا كان لديك ملف نصي يحتوي على سلسلة المحارف:

WWWWWWBWWWWWWBBBWWWW

سنقوم بترميزها: 6W1B6W3B4W (أي نضع الحرف وبجانبه تكراراته)

نلاحظ أنه تم ضغط 20 حرف الى 10 أرقام وحروف.

- تكون هذه الخوارزمية فعالة عندما يكون لدي تكرار كبير ومتتالي ولكن عندما لا يوجد لدي تكرارات متتالية تصبح هذه الخوارزمية غير فعالة.
- مثال: إذا كانت لديك الرسالة:

ABCD → 1A1B1C1D

نلاحظ أن حجم الرسالة الأصلية أصغر أي إرسالها قبل الترميز يكون أفضل.

ملاحظة: إذا كان لدي ملف يحتوي على قيم مكررة ولكن هذه القيم غير متتالية فإن هذه الخوارزمية غير مفيدة بالنسبة لنا.

إيجابياتها:

1. خوارزمية بسيطة.
2. خوارزمية lossless لا تفقد بيانات
3. عملية الضغط وفك الضغط تكون سريعة
4. تعمل بشكل جيد مع البيانات التي يكون فيها تكرارات كثيرة ومتتالية

سلبياتها:

1. تكون غير فعالة مع البيانات التي لا تحتوي تكرارات أو العشوائية.
2. بسيطة جداً؛ حيث أنها لا تستفيد من الأنماط المعقدة أو التركيبات الأكثر تطوراً الموجودة في معظم البيانات.

Information theory

- هي فرع من الرياضيات نشأت عام 1940 s
- هذه النظرية تقول أن ضغط البيانات يهتم بالزيادة عن الحاجة لذلك يدخل الضغط في مجال نظرية المعلومات أي إذا تمكنا من التخلص من الزيادة عن الحاجة نكون قد قللنا حجم الرسالة .

وهنا يتبادر إلى ذهننا سؤال:

إذا كانت لدي رسالة محددة وقمت بضغطها ما هو أصغر حجم ممكن أن نصل إليه؟

- يمكن أن نضغط رسالة ونجد أن هناك خوارزمية أخرى ممكن أن تضغطها بطريقة أفضل فيجب أن نجد مقياس لنقول أن هذه الخوارزمية فعالة أم لا فإذا كانت فعالة لا نلجأ إلى خوارزمية أخرى
- ومن هنا ظهر مفهوم بالرياضيات يخبرني ما هو أصغر حجم للضغط لرسالتي أي ينتج لدي رقم يدعى entropy ونقارن معه إذا ضغطنا برقم قريب منه نستنتج أن هذه الخوارزمية فعالة أم لا.

Information of symbol

لديك الرسالة:

$$message = \{a_1, a_2, a_3, \dots, a_n\}$$

حيث:

$P(a_i)$ هو احتمال وقوع الرمز a_i وهو عدد مرات ووروده بالرسالة على طول الرسالة.

يكون مجموع احتمالات ورود الحرف المكونة للرسالة يساوي الواحد:

$$P(a_1) + P(a_2) + P(a_3) + \dots + P(a_n) = 1$$

مثال: لدي الرسالة:

aaaaabcccd

فإن:

$$P(a_i) = \frac{\text{عدد مرات ورود الحرف } a_i}{\text{عدد أحرف الرسالة الكلي}}$$

$$\inf(a_i) = -\log_2(P(a_i))$$

نعرف $\inf(a_i)$: هو مدى أهمية الحرف في الرسالة فإذا كان الحرف مكرراً كثيراً تكون أهميته قليلة (أي أنه لا يملك

معلومات مهمة وفريدة وأستطيع الاستغناء عنها) على عكس الأحرف التي لا تكون مكررة بشكل كبير فتكون أهميتها

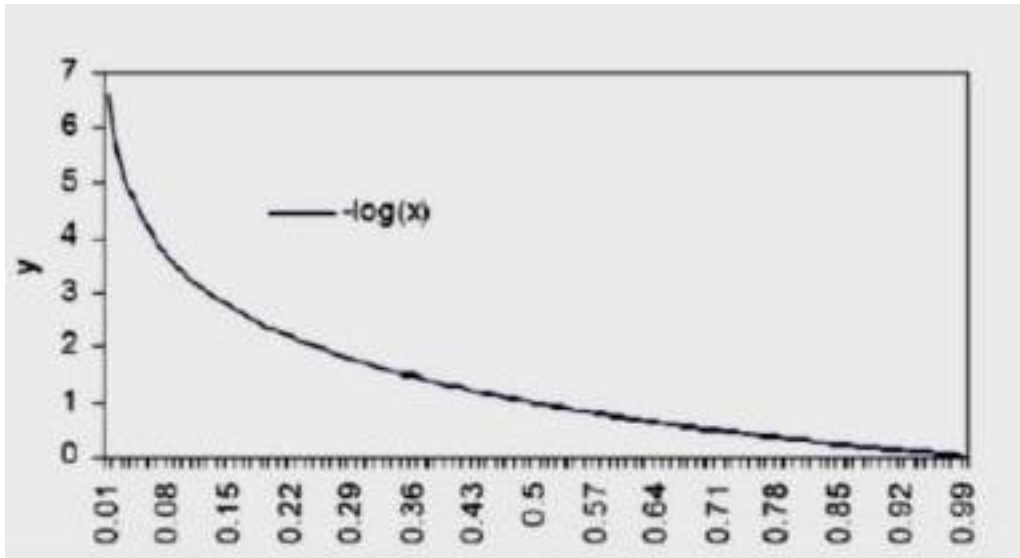
كبيرة وتمتلك معلومات نوعية وفريدة ولا أستطيع الاستغناء عنها

- يعطينا هذا القانون مقدار المعلومات التي يحملها الحرف الواحد (الرمز).

ملاحظة: دائماً القيمة الاحتمالية تكون بين 0 و 1.

- Log لقيمة أصغر من 1 يكون سالب لذلك نضرب بـ - لينتج لدينا مقدار موجب.

ملاحظة هامة: كلما كان ورود الحرف في الرسالة أكثر كلما قلت أهميته وقل مقدار المعلومات التي يحملها وتصبح ال Info أقل وسوف يتمثل على عدد بتات أقل.



حيث يمثل المحور x احتمال الرمز a_i أي $P(a_i)$ ويمثل المحور y أهمية المحرف a_i أي $inf(a_i)$

■ Entropy :

- الإنتروبي للبيانات : هو مجموع قيم الإنتروبي لكل رمز من رموزها ويعرف على مستوى الرسالة كاملة.
- أي نأخذ أهمية الحرف باحتمال وروده في الرسالة.
- الإنتروبي هو ببساطة متوسط مقدار المعلومات التي يحملها كل رمز، ويمثل عدد البتات المعلوماتية في الرسالة.
- يحدد الحد الأدنى لمتوسط عدد البتات اللازمة لترميز كل رمز.

$$\eta = \sum_{i=1}^n p(a_i) \log_2\left(\frac{1}{p(a_i)}\right) = - \sum_{i=1}^n p(a_i) \log_2 p(a_i)$$

أي نأخذ أهمية الحرف باحتمال وروده في الرسالة

مثال: إذا كان لدي رسالة تحتوي على الأحرف {a, b, c}

حيث:

$$P(a) = \frac{1}{8}, \quad p(b) = \frac{1}{4}, \quad p(c) = \frac{5}{8}$$

ملاحظة: ممكن أن يعطينا الرسالة ونحن نقوم بحساب الاحتمالات

نطبق قانون الانتروبي على كل قيمة ومن ثم نقوم بجمع القيم:

■ نطبق على الحرف a:

$$\frac{1}{8} * -\log_2\left(\frac{1}{8}\right) = \frac{1}{8} * 3$$

▪ نطبق على الحرف b:

$$\frac{1}{4} * -\log_2\left(\frac{1}{4}\right) = \frac{1}{4} * 2$$

▪ نطبق على الحرف c:

$$\frac{5}{8} * -\log_2\left(\frac{5}{8}\right) = \frac{5}{8} * 0.68$$

▪ نجمع القيم الثلاثة:

$$\eta = \frac{1}{8} * 3 + \frac{1}{4} * 2 + \frac{5}{8} * 0.68 = \underline{1.3\text{bits}}$$

الرقم الناتج 1.3 هو أصغر حجم بتات لكل حرف وبالتالي يجب ألا نحصل على رقم أقل من 1.3 للحرف الواحد

▪ إذا احتوت رسالتي على 8 أحرف فيكون $1.3 * 8$ وهو أصغر حجم لرسالتي أي الناتج 10.4 وهو الحجم بعد الضغط

▪ مجرد معرفة هذا الرقم لا نلجأ للبحث عن خوارزمية أفضل من الخوارزمية التي استخدمتها، (مهما كانت الخوارزمية المستخدمة ما يهمنا هو الحصول على حجم للرسالة قريب من هذا الرقم، أي إذا كان حجم الرسالة بعد الضغط قريب من هذا الرقم لا نلجأ للبحث عن خوارزمية أفضل).

▪ **في أفضل حالة:** يكون لدي حرف واحد داخل الرسالة أي أن الرسالة تحتوي فقط على الحرف a والحرف b، c غير موجودين في الرسالة أي احتمالهما 0

$$H = -1 * \log_2(1) = 0$$

وهو حجم الرسالة بعد الضغط الجديد ولكن هذا نظريا (هذه هي الحالة المثلى ولا يمكن الوصول إليها)

▪ **في أسوأ حالة:** تكون الأحرف جميعها بنفس الأهمية ولا يوجد أحرف مكررة
مثال:

لدينا رسالة في 3 أحرف {a, b, c} ونفرض احتمال ظهور الأحرف:

$$P(a) = P(b) = P(c) = \frac{1}{3}$$

فإن:

$$\eta = \sum_{i=1}^n p(a_i) \log_2\left(\frac{1}{P(a_i)}\right)$$

$$\eta = \frac{1}{3} \log_2(3) + \frac{1}{3} \log_2(3) + \frac{1}{3} \log_2(3)$$

$$\eta = 3 \times \frac{1}{3} \log_2(3) = \underline{1.6 \text{ bit}}$$

وهو أصغر حجم بتات لكل حرف



No Cost
Too Great

Shannon-Fano coding

- تستخدم هذه الخوارزمية إذا كان لدينا n من الرموز مع احتمالات حدوث معروفة لكل منها.
- كما نعلم سابقاً أن كل حرف يمثل على 1Byte في الـ ascii code ويوجد لدي 256 حالة ولكن رسالتي لا تحتوي على جميع الأحرف لماذا لنحجز ونستهلك من الذاكرة حجم أكبر من الحجم الذي نحتاجه؟!
- لذلك سنخرج من هذه الفكرة أن كل حرف يمثل على 1B ويمكننا أن نضع لكل حرف على عدد بتات تناسبه.

مثال: لدي الرسالة:

A	B	C	D
00	01	10	11

أي يمكن أن أضع كل حرف على 2bit تلقائياً حجم الرسالة سيقبل ونحصل على ترميز جديد للرسالة ولكن يوجد لدي مشكلة كيف أفك ضغط هذه الرسالة؟

عند فك الضغط:

- يجب أن نقرأ كل 2bit معاً لأن كل حرف رمزناه على 2 bit.
- يجب معرفة كل حرف وترميزه.
- طالما أننا لم نعد نرمز على 8bits هذا يعني أننا قللنا الحجم.

* لتطبيق هذه الخوارزمية:

1. نرتب الاحتمالات ترتيب تنازلي.
2. نقسم مجموعة الرموز إلى مجموعتين فرعيتين بحيث يكون مجموع الاحتمالات في كل قسم مساو لمجموع الاحتمالات في القسم الثاني أو قريب منه جداً.
3. نخصص البتات:

- نضيف الرقم 0 لترميز المجموعة الفرعية الأولى.
 - نضيف الرقم 1 لترميز المجموعة الفرعية الثانية.
- ونكرر حتى لا يتبقى لدينا أي مجموعات جزئية ولا نستطيع القيام بتقسيم الرموز

ملاحظة: في النهاية ينتج لدي حرف واحد داخل كل بلوك.

في المثال السابق رمزنا كل حرف على 2bits لأن الأحرف جميعها متساوية بالأهمية، أما إذا كانت الأحرف غير متساوية بالأهمية كالمثال التالي:

aaaa BB aaa CC aa

- نلاحظ أن الحرف a تكرر كثيراً أي أن أهميته قليلة لذلك نعطيه أقل عدد من البتات مثلاً 1bit فيكون الرمز الخاص بالحرف a هو 0
- B ترميزه 01 يجب أن أعدل ترميزه لأن a أصبح ترميزه 0 نعدل ترميزه ويمكن أن نضيف عدد بتات أكثر له.

ملاحظة: يجب أن يكون مجموع الاحتمالات بعد التقسيم إما متساوي أو متقارب ونكرر عملية التقسيم حتى لا يبقى لدي أي قيم.

مثال: لدي رسالة تحتوي على الحروف: $\{a_1, a_2, a_3, a_4, a_5, a_6, a_7\}$ واحتمال ورودهم على الترتيب $\{0.25, 0.20, 0.15, 0.15, 0.10, 0.10, 0.05\}$

ونريد استخدام خوارزمية Shannon-Fano Coding للضغط:

- نرتب الأحرف تنازلياً حسب الاحتمال:
 $a_1 = 0.25, a_2 = 0.20, a_3 = 0.15, a_4 = 0.15, a_5 = 0.10, a_6 = 0.10, a_7 = 0.05$
- نقسم هذه الرموز إلى مجموعتين لها احتمالات متساوية تقريباً ونضع بجانب الرموز التي تقع فوق خط التقسيم 1 وبجانب الرموز التي تقع تحت خط التقسيم 0
- إذا قسمنا بين a_1, a_2 نحصل على مجموعتين:
 - المجموعة الأولى تحتوي a_1 واحتماله 0.25
 - والمجموعة الثانية تحتوي بقية الرموز ومجموع الاحتمالات هو 0.75
 هذا يعني أن هذا القسم غير صحيح لوجود فرق كبير بين الاحتمالات.
- إذا قسمنا بين a_2, a_3 نحصل على مجموعتين:
 - الأولى: تحوي a_1, a_2 ومجموع احتمالاتها 0.45.
 - الثانية: تحوي بقية الرموز ومجموع احتمالاتها 0.55
 نستنتج أن الاحتمالات قريبة من بعضها لذلك سوف نستخدم هذه القيم.

a_1	0.25	<u>1</u>
a_2	0.20	<u>1</u>
a_3	0.15	0
a_4	0.15	0
a_5	0.10	0
a_6	0.10	0
a_7	0.05	0

نلاحظ أننا وضعنا بجانب الرموز التي تقع فوق خط التقسيم 1 والرموز التي تقع تحت خط التقسيم 0 وضعنا بجانبها 0

- نقسم المجموعتين اللتين حصلنا عليهن إلى مجموعات أيضاً مع الحفاظ على الشرط أن تكون الاحتمالات متساوية أو قريبة من بعضها

■ نقسم بين a_1, a_2 :

a_1	0.25	<u>1</u>	<u>1</u>
a_2	0.20	<u>1</u>	0
a_3	0.15	0	0
a_4	0.15	0	0
a_5	0.10	0	0
a_6	0.10	0	0
a_7	0.05	0	0

■ نلاحظ أن التقسيم المناسب هو بين a_4, a_5 لأن مجموع الاحتمالات قريبة من بعضها أي 0.30 و 0.25

a_1	0.25	<u>1</u>	<u>1</u>	
a_2	0.20	<u>1</u>	0	
a_3	0.15	0	0	<u>1</u>
a_4	0.15	0	0	<u>1</u>
a_5	0.10	0	0	0
a_6	0.10	0	0	0
a_7	0.05	0	0	0

■ الخلايا عند a_1, a_2 تبقى فارغة لأنه لم يعد لدينا أي تقسيم بينهما

■ نلاحظ أن التقسيم المناسب هو بين a_3, a_4 لأن مجموع الاحتمالات قريبة من بعضها 0.60, 0.40

<u>a_1</u>	<u>0.25</u>	<u>1</u>	<u>1</u>		
a_2	0.20	<u>1</u>	0		
<u>a_3</u>	<u>0.15</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>1</u>
a_4	0.15	0	0	<u>1</u>	0
a_5	0.10	0	0	0	0
a_6	0.10	0	0	0	0
a_7	0.05	0	0	0	0

■ نلاحظ أن التقسيم المناسب بين a_5, a_6 حيث الاحتمالات 0.10 و 0.15

<u>a_1</u>	<u>0.25</u>	<u>1</u>	<u>1</u>			
a_2	0.20	<u>1</u>	0			
<u>a_3</u>	<u>0.15</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>1</u>	
a_4	0.15	0	0	<u>1</u>	0	
a_5	0.10	0	0	0	0	<u>1</u>
a_6	0.10	0	0	0	0	0
a_7	0.05	0	0	0	0	0

وأخيراً نقسم بين a_6, a_7 :

a_1	0.25	1	1				
a_2	0.20	1	0				
a_3	0.15	0	0	1	1		
a_4	0.15	0	0	1	0		
a_5	0.10	0	0	0	0	1	
a_6	0.10	0	0	0	0	0	1
a_7	0.05	0	0	0	0	0	0

نلاحظ أنه لم يعد لدينا أي تقسيم وحصلنا على الترميز النهائي لكل رمز

الرمز	الاحتمال	الترميز النهائي			
a_1	0.25	1	1		
a_2	0.20	1	0		
a_3	0.15	0	1	1	
a_4	0.15	0	1	0	
a_5	0.10	0	0	1	
a_6	0.10	0	0	0	1
a_7	0.05	0	0	0	0

نلاحظ ان: الترميز النهائي للرموز هو:

$a_1 = 11$, $a_2 = 10$, $a_3 = 011$, $a_4 = 010$, $a_5 = 001$, $a_6 = 0001$, $a_7 = 0000$

نلاحظ أن الحرف الذي تكرر كثيراً (احتمال ظهوره كبير) يمثل بعدد بتات أقل من الحروف ذو احتمال ظهور أقل

ملاحظة: عندما نريد فك ضغط الرسالة يجب أن نعرف تمثيل كل حرف، فعندما نعرف **11** نعلم أن هذا الحرف هو a_1 وهكذا..

مثلاً إذا أتى لدينا الترتيب التالي للأحرف:

A2	A1	A7	A6
10	11	0000	0001

نضع تحت كل حرف ترميزه.

سؤال: كيف أتمكن من فك هذه الرسالة بعد الضغط؟

نقرأ الترميزات بت بت ولا نقرأ كل 2bit لأن بعض المحارف ترمز على 3bit وبعضها على 2bit
إذا أتانا الترميز التالي 10110000000

- نقرأ البت الأول 1 نبحث في الأحرف هل يوجد حرف ترميزه 1؟ لا لا يوجد.
- نكمل في القراءة فنجد 10 هل يوجد حرف ترميزه 10؟ نعم الحرف a_1 ترميزه 10 هكذا نكون حصلنا على أول ترميز هو a_1 ونكمل بنفس الطريقة لنحصل على باقي الرموز.
- نستمر بالقراءة حتى نحصل على رمز وإذا لم نحصل على رمز يكون لدي خطأ بالضغط.
- قرأنا بت بت لأنه لا يوجد لدي رمزين يمتلكان البداية نفسها Prefix وهي أن يكون لدي رمز هو جزء من رمز ثاني.

ملاحظة: إذا كان لديك الرمز 11 لا يمكن لرمز آخر أن تكون بدايته 11 إذا تحققت هذه القاعدة نك بال طريقة السابقة.

ممك أن يعطينا في التمرين الأحرف واحتمالاتها ونحن نحسب الرموز، أو الأحرف مع الرموز سيعطيني إياها في الهيدر (Header) قبل إرسال الـ الداتا.

ملاحظة: الهيدر يخزن الرموز مع نوع الخوارزمية.

ملاحظة: عادةً تقع لدي أخطاء أثناء النقل أما بالضغط قليلاً ما يحدث أخطاء ويمكن أن يتغير أثناء النقل عدة رموز، ولا نتمكن من فك الضغط وعادةً يعطينا خيار لإصلاح الملف قد يكون حل مناسب ولكنه ليس الحل الأمثل لأننا سنخسر جزء من الداتا لكن **المشكلة الحقيقية** عندما يتغير لدي البت الموجود بالهيدر لأنني لا أستطيع فك الملف كاملاً ولا يمكن حل المشكلة.

■ **مثال 2:** لدينا الرموز التالية مع احتمالات كل منها:

■ نقسم بين a_2, a_3 :

الرمز	الاحتمال	
a_1	0.25	1
a_2	0.25	1
a_3	0.125	0
a_4	0.125	0
a_5	0.125	0
a_6	0.125	0

■ نقسم بين a_1, a_2 :

الرمز	الاحتمال		
a_1	0.25	1	1
a_2	0.25	1	0
a_3	0.125	0	0
a_4	0.125	0	0
a_5	0.125	0	0
a_6	0.125	0	0

■ نقسم بين a_4, a_5 :

الرمز	الاحتمال			
a_1	0.25	1	1	
a_2	0.25	1	0	
a_3	0.125	0	0	1
a_4	0.125	0	0	1
a_5	0.125	0	0	0
a_6	0.125	0	0	0

■ نقسم بين a_3, a_4 :

الرمز	الاحتمال				
a_1	0.25	1	1		
a_2	0.25	1	0		
a_3	0.125	0	0	1	1
a_4	0.125	0	0	1	0
a_5	0.125	0	0	0	
a_6	0.125	0	0	0	

■ نقسم بين a_5, a_6 :

الرمز	الاحتمال					
a_1	0.25	1	1			
a_2	0.25	1	0			
a_3	0.125	0	0	1	1	
a_4	0.125	0	0	1	0	
a_5	0.125	0	0	0	0	1
a_6	0.125	0	0	0	0	0

■ نلاحظ أنه لم يعد لدينا أي تقسيم وحصلنا على الترميز النهائي لكل رمز

الرمز	الاحتمال	الترميز النهائي		
a_1	0.25	1	1	
a_2	0.25	1	0	
a_3	0.125	0	1	1
a_4	0.125	0	1	0
a_5	0.125	0	0	1
a_6	0.125	0	0	0

■ نلاحظ أن الترميز النهائي:

$a_1 = 11$, $a_2 = 10$, $a_3 = 011$, $a_4 = 010$, $a_5 = 001$, $a_6 = 000$

Huffman coding

هي خوارزمية مشابهة لخوارزمية Shannon-Fano Coding ولكنها تعتمد على بناء شجرة ثنائية للاحتفاظ بالترميز ولتحديد تمثيل كل حرف من الأحرف الموجودة في الـ data المراد ضغطها.

خطوات إنشاء الشجرة الثنائية:

1. تتم عملية البناء من الأوراق صعوداً إلى الأعلى.
2. نعتبر كل رمز مع احتمالية ظهوره شجرة مستقلة.
3. يتم دمج الشجرتين اللتين تمتلكان عندها أقل الترددات (مع جمع هذه الترددات معاً).
4. تكرر هذه العملية حتى تبقى شجرة واحدة فقط .
5. بدءاً من الجذر وصولاً إلى الأوراق نضيف الرقم 1 إلى كود الفرع الأيمن للشجرة و 0 إلى كود الفرع الأيسر.

■ مثال: ليكن لدينا الجدول التالي الذي يعبر عن الاحرف الموجودة ضمن رسالة نريد ضغطها:

المساحة التي يشغلها	احتمال وروده	عدد مرات تكراره	الحرف
56 bits	0.097	7	"W"
96 bits	0.166	12	"U"
120 bits	0.208	15	"X"
144 bits	0.25	18	"V"
160 bits	0.277	20	"Y"
576 bits	1	72	مجموع

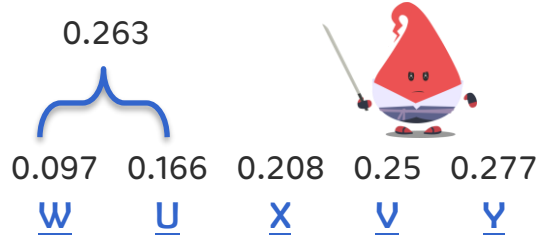
سنقوم ببناء شجرة أوراقها هي الرموز W,U,X,V,Y حيث يكون لكل ورقة مسار من الـ Root إلى هذه الورقة

خطوات بناء الشجرة:

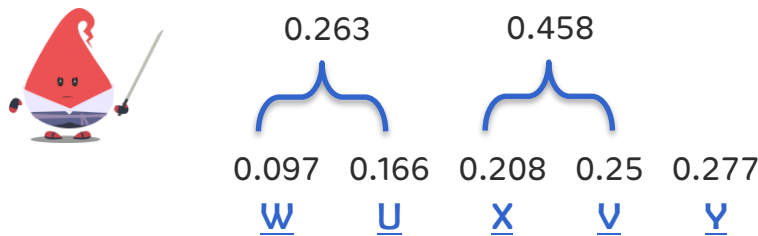
1. نقوم بترتيب الرموز حسب الاحتمالات ترتيباً تصاعدياً:

0.097	0.166	0.208	0.25	0.277
W	U	X	V	Y

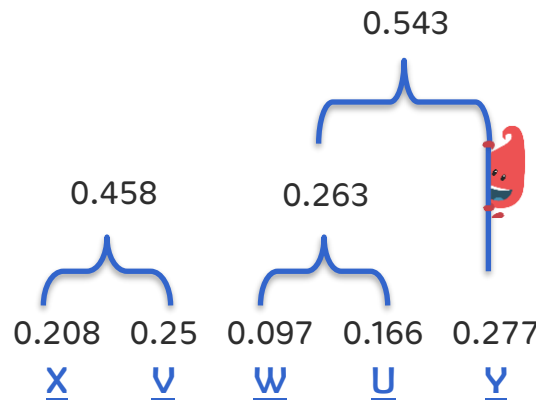
2. نركب أصغر احتمالين منهم ونقوم بدمجهما ووضع الاحتمال الجديد الناتج عن مجموع احتماليهما:



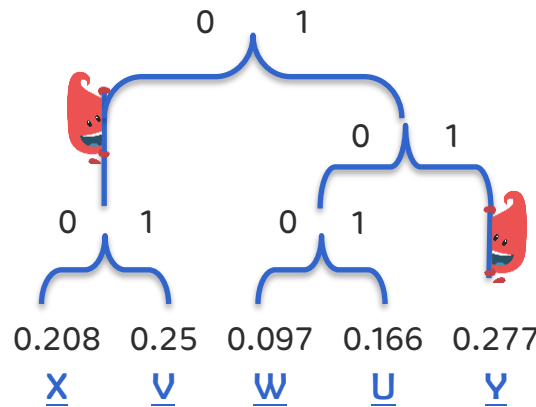
3. نكرر الخطوة السابقة:



4. نكرر الخطوة السابقة أيضاً حتى نصل إلى ال root :



5. وبما أننا وصلنا للجذر نعطي كل فرع يميني 1 وكل فرع يساري 0 بدءاً من ال root :



نقرأ الرموز من الجذر إلى الأوراق (أي من الأعلى للأسفل)

وبالتالي يصبح لدينا الترميز النهائي للحروف:

$X = 00$, $V = 01$, $W = 100$, $U = 101$, $Y = 11$

نقوم بضرب احتمال كل رمز بطوله (أي عدد بتات ترميز التي حصلنا عليها) ونجمع النتائج فنحصل على متوسط طول الأحرف:

$$0.208 \times 2 + 0.25 \times 2 + 0.097 \times 3 + 0.166 \times 3 + 0.277 \times 2 = 2.259 \text{ bit}$$

لنحصل على الحجم الكلي نقوم بضرب كل حرف بتكراره:

نلاحظ من الجدول السابق أن الحرف U تكرر 12 مرة وهو مرمز على 3 bits فنضرب 3×12

وكذلك مع بقية الأحرف ونجمع النتائج لنحصل على الحجم الكلي:

$$12 \times 3 + 18 \times 2 + 7 \times 3 + 15 \times 2 + 20 \times 2 = 163 \text{ bits}$$

وهو حجم الرسالة بعد الضغط.

■ نجد نسبة الضغط:

$$\text{Compression Ratio} = 1 - \frac{163}{576} = 71.7\%$$

أي أنه تم ضغط الملف بنسبة 71%.

ملاحظة 1: جميع خوارزميات الضغط تستهلك وقت ولكن نحصل على حجم أصغر لتخزين هذا الملف على وسيط تخزيني.

ملاحظة 2: الوقت الأكبر يكون بحساب احتمال كل حرف في الملف لأننا بحاجة أن نمسح الملف بالكامل لنحسب احتمال ورود كل حرف بداخله إذا حصلنا على الاحتمالات فيصبح لدينا بناء الشجرة لا يستهلك وقت.

■ خصائص خوارزمية Huffman :

1. لا يوجد غموض في عملية فك الترميز البيانات المشفرة توجد طريقة واحدة لفك تشفيرها (حيث يتم إرسال شجرة هافمان بالكامل والـ decoder يقوم ببناء الشجرة ومن ثم يقوم بفك ترميز أكواد هافمان).
2. أكبر مساوئها أنها تتطلب معرفة مسبقة بإحصائيات حول تكرار الأحرف والاحتمالات.
3. عدد البتات الوسطي لتمثيل الحرف في خوارزمية Huffman هو أصغر تماماً من $\eta + 1$ حيث η هو الـ Entropy أي أن:

$$\eta \leq \text{average code length} \leq \eta + 1$$

أستطيع من خلال هذا القانون أن أتوقع حجم أي رسالة (ماذا سيصبح حجمها بعد الضغط) نحسب الـ Entropy ونضيف عليه 1 ويكون هو متوسط عدد البتات.

■ مشاكل الحلول الإحصائية:

يجب علينا قراءة الملف بأكمله لنقوم بضغطه ولكن إذا لم يكن الملف كاملاً موجوداً لدي ما الحل؟
مثلاً: إذا كان لدي بث مباشر أو مباراة رياضية وأريد أن أضغطها، لا يمكنني أن أنتظر حتى تنتهي المباراة لأنشئ ملف وأضغطه لأنه بث مباشر والمشاهدين يريدون رؤيته مباشرة.
 ولحل هذه المشكلة ظهرت لدي خوارزمية [adaptive Huffman Coding](#).

قد تنتهي الأسباب، لكن رحمة الله لا تنتهي، وقدرته لا يعجزها شيء



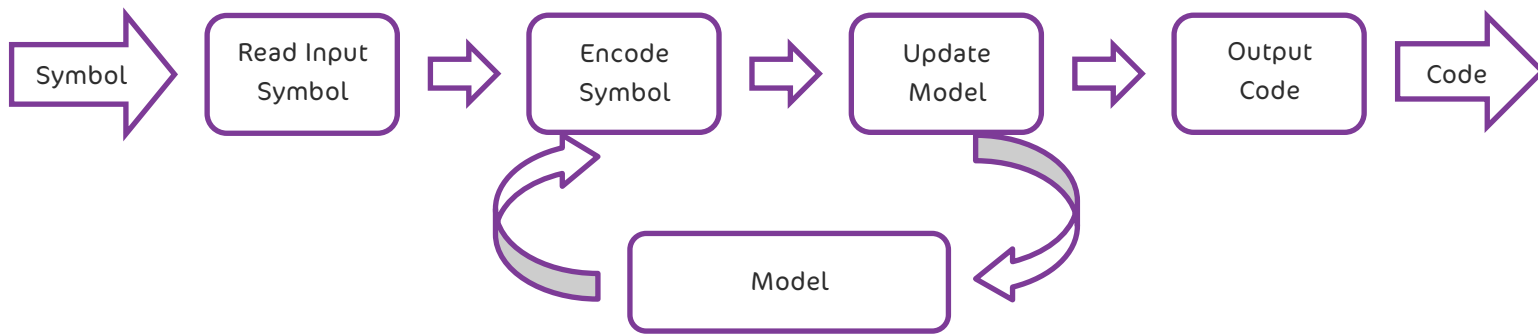
adaptive Huffman Coding

في هذه الخوارزمية يكون لدي Stream من البيانات ونضغط هذه الـ Stream بطريقة إحصائية.

■ كيف أستطيع أن أضغط بطريقة إحصائية والبث قيد التشغيل ؟

- نفعل عل الإحصائية بحيث إذا أتى لدي حرف تتغير هذه الإحصائية أي أن المعلومات الإحصائية عن الـ data في هذه الخوارزمية تتجمع وتحديث بشكل ديناميكي أثناء قراءة الـ data على عكس خوارزمية الـ Huffman العادية التي تتطلب معرفة مسبقة بالمعلومات الإحصائية.
- نقوم بإنشاء شجرة للضغط أثناء قراءة الداتا التي نريد ضغطها وعند قراءة الحرف نقوم بتمثيله بالـ ASCII إذا كان أول مرة يظهر فيها أما إذا كان الحرف موجود مسبقاً نمثله حسب الشجرة التي قمنا ببنائها ثم نقوم بتحديث الشجرة فإذا كان الحرف جديد فنقوم بإضافته إلى الشجرة أما إذا كان الحرف موجود مسبقاً فنقوم بتحديث احتمالية ظهوره في الشجرة.
- نلاحظ أن الحرف الواحد يختلف على حسب موقعه في الداتا فإذا جاء حرف معين في أول الداتا سوف يكون تمثيله مختلف عن تمثيل الحرف نفسه عند وروده في آخر الداتا لأننا نقوم بتعديل الشجرة باستمرار وهذا يؤدي إلى اختلاف تمثيل الحرف نفسه.

ملاحظة: كلما أتانا حرف سنضغطه ونرسله وليس لدي buffer للتخزين فيها.



ولفك الضغط نقوم بعمل نفس العملية ولكن بشكل عكسي ثم نقوم بتحديث الشجرة لمعرفة التمثيل الجديد للأحرف المضغوطة.

إذا صدقت النية، يسّر الله الطريق، ولو بدا في أوله صعباً

Encoder

```
Initial_Code();
while not EOF {
    get(c);
    encode(c);
    update_tree(c);
}
```

Decoder

```
Initial_Code();
while not EOF {
    decode(c);
    output(c);
    update_tree(c);
}
```

1. دالة ال (Initial_Code) الأولي:

تقوم بربط كل حرف مع كود ال (ASCII) الخاص به أو أي كود متفق عليه مسبقاً.

أي أن A مثلاً تمثيله بالـ ASCII هو (01000001)

يبدأ ال Encoder وال Decoder بشجرة فارغة ثم يقومون بالتعديل عليها عند قراءة كل حرف.

- أول حرف يكتب بدون ضغط أي يكتب بصيغة ال (ASCII) ثم يضاف هذا الحرف إلى الشجرة ويسند إليه ترميز معين (أي يتغير ترميزه من ASCII إلى رمز تعطيه إياه الشجرة) وعند ورود هذا الحرف مرة أخرى يكتب بالرمز الذي أسند إليه بالشجرة.

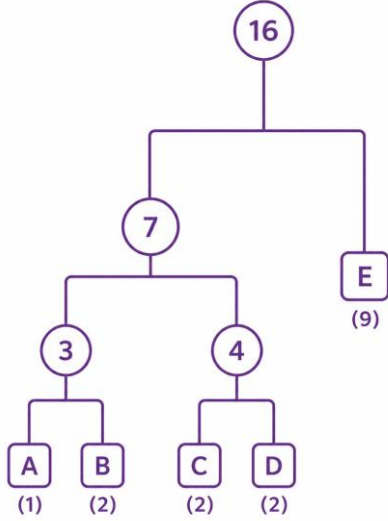
2. دالة ال (update tree):

نقوم بأخذ الحرف المقروء وإذا كان حرف جديد تضيفه إلى الشجرة أما إذا كان موجود مسبقاً فنقوم بزيادة عداد ظهوره وتقوم بتعديل شكل الشجرة إذا لزم الأمر وعندها يختلف ترميز الأحرف الموجودة فيها نقوم بإجراء (Swap) يمين أفرع الأشجار في حال أصبح فرع يساري ما أكبر من فرع يميني.

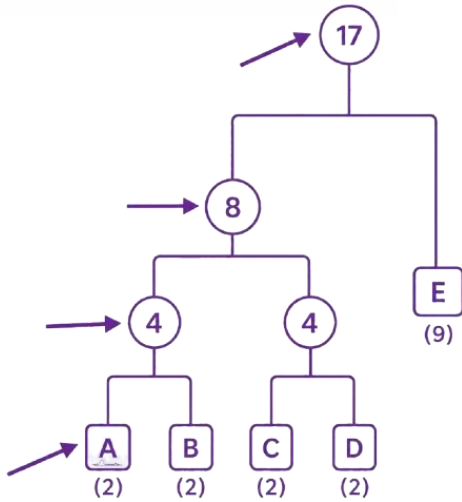
لا تحكم على البداية بصغرها، فالأشجار العظيمة بدأت ببذرة

ليكن لدينا هذه الشجرة في لحظة ما أثناء قراءتنا لل data تكون هذه الشجرة عند ال Encoder وال Decoder:

- الجذر العلوي (16) يمثل الوزن الإجمالي لكل الحروف المقروءة في هذه اللحظة وهو مجموع الفرع الأيمن والأيسر
- العقد الداخلية تمثل مجموع أوزان الحروف التي تقع أسفلها مباشرة العقدة (3) هي مجموع تكرارات A+B وهكذا
- الأوراق تمثل الحروف التي تمت قراءتها حتى الآن
- الأرقام تحت الأوراق تمثل عدد مرات تكرار كل حرف أي أن الحرف A تكرر مرة واحدة والحرف B تكرر مرتين وهكذا....



ولنفرض أنه بعد هذه اللحظة قرأنا حرف "A" فتصبح الشجرة:



- السهم النهدي يشير إلى الحرف الذي تم قراءته في هذه اللحظة تماماً
- نلاحظ من المخطط أن السهم تحت الحرف A وتغير من 1 إلى 2 فهذا يعني أن الخوارزمية استلمت الآن حرف A جديد

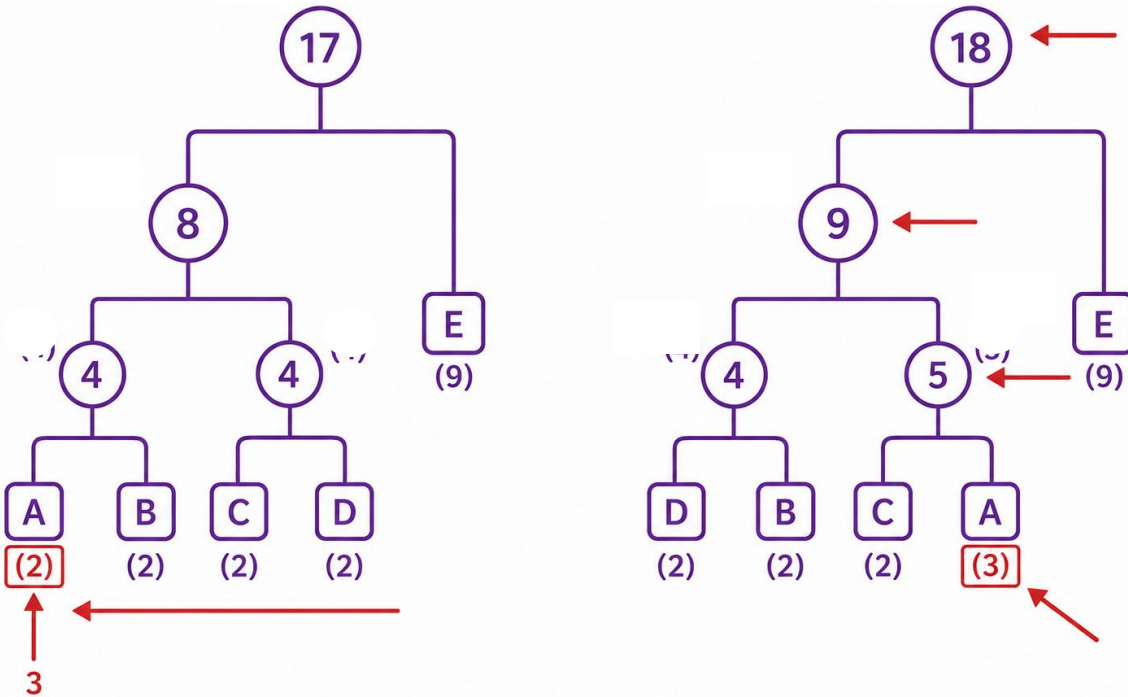
ملاحظة: عند زيادة تكرار حرف معين يجب تحديث الشجرة من الأسفل إلى الأعلى للحفاظ على خاصية الترتيب، حيث يجب أن تكون الأوزان مرتبة تصاعدياً من اليسار إلى اليمين ومن الأسفل للأعلى

أقصى الأيام قد تكون آخر ما يفصل بينك وبين الفرغ

عند تحول وزن A إلى 2:

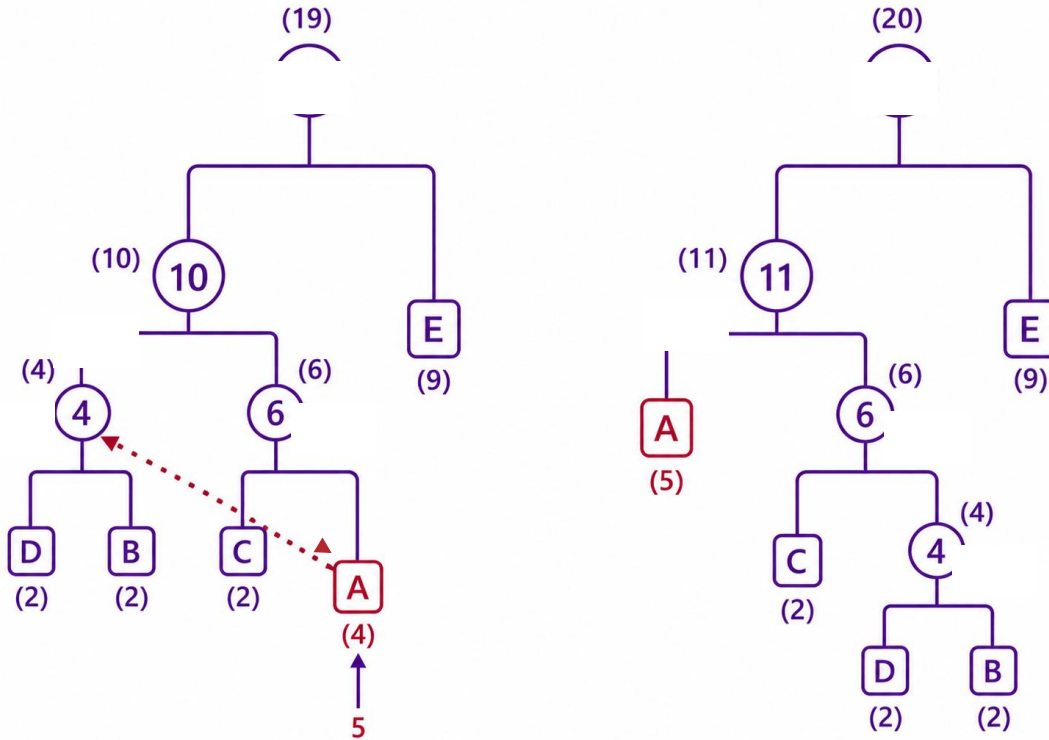
1. نقوم بتحديث عقدة الأب مباشرة: تصبح $4=2+2$ بدلاً من 3
 2. نفحص الترتيب وتبادل الأماكن (Swap): بما أن الفرع الأيسر أصبح يحمل الوزن (4) والفرع الأيمن مجاوره (4)، ستتحقق الخوارزمية من أرقام العقد (Node numbers) إذا وُجد أن هناك عقدة بوزن أقل تقع في ترتيب أعلى أو يساري خاطئ، سيتم عمل تبديل (Swap) للأماكن (مثل تبديل الفرع بالكامل أو الأحرف) لإعادة توازن الشجرة.
 3. تحديث الجذور العليا: يستمر التحديث صعوداً، لتصبح العقدة (7) مساوية لـ $8=4+4$ وبالتالي يصبح الجذر الإجمالي في الأعلى 17 بدلاً من 16
- نلاحظ أنه لا حاجة لإعادة ترتيب الشجرة

■ وفرضاً قرأنا حرف "A" مرة أخرى فتصبح الشجرة بالشكل الآتي:



- نلاحظ أن عدد مرات تكرار "A" أصبح (3) أي أصبح أكبر من عدد مرات تكرار B, C, D لذلك قمنا بتعديل الشجرة ووضعناه بدال الـ "D" حتى تبقى الأوزان مرتبة تصاعدياً من اليسار إلى اليمين ومن الأسفل للأعلى
- نلاحظ أن ترميز "A" الحالي أصبح "A=011" (نقرأ الترميز من الجذر للأوراق كما تعلمنا سابقاً حيث أن كل فرع يميني نضع عنده 1 والفرع اليساري نضع عنده 0) وقد كان قبل ذلك "A=000".

■ ولنفرض أنه قرأنا حرفين "A" أي ازداد عدد مرات التكرار مرتين فتصبح الشجرة بالشكل التالي:



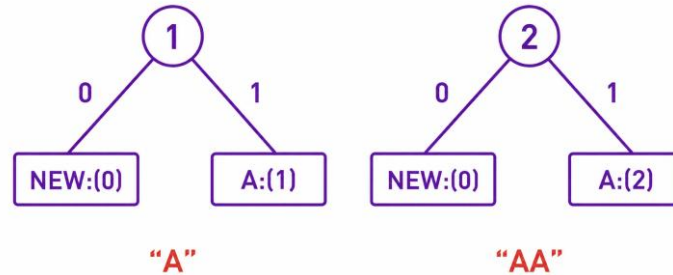
نلاحظ أن تكرار A أصبح 5 والعقدة الداخلية فوقها 4 ولكن شرط الشجرة أن تبقى مرتبة من الأسفل للأعلى لذلك نقوم بعملية SWAP بينهما أي سوف تصعد A إلى الأعلى مع وزنها ويبقى لدينا B, C, D

عند فك الضغط:

- **يجب أن نميز:** إذا كان الحرف الذي ن فك ضغطه هو حرف غير مضغوط (ASCII) أي أنه لم يمر من قبل، أو أن الحرف مضغوط وممرز حسب شجرة الضغط
- ومن أجل حل هذه المشكلة نضيف على شجرة التشفير عقدة (new)
- حيث أن كل حرف جديد مرمز بال (ASCII) نضع عقده ال (new) قبله عند الضغط وعند فك الضغط عندما نقرأ ال (new) نعلم أن الحرف الذي سيأتي بعدها مرمز بال (ASCII).
- وعقدة ال (new) في الشجرة دائماً يكون عدد مرات تكرارها يساوي "0" أي لا نزيده وتكون موجودة في الشجرة عند ال Encoder و ال Decoder قبل البدء بالقراءة.

إذا ضاقت بك الدنيا، فتذكر أن رب الدنيا لا تضيق خزائنه

- عندما نبدأ القراءة عند الـ Encoder نضع أولاً الرمز "new" و "0" ثم نضع ترميز الـ (ASCII) الخاص بالحرف الذي قرأناه ثم نضيف الحرف للشجرة.
- نفرض أن أول حرف قرأناه هو "A" وترميزه هو 01000001 فتصبح لدينا الشجرة:
لدينا السلسلة الآتية ونريد قراءتها AADCCD



- تبدأ الشجرة دائماً بعقدة فارغة تسمى NEW ووزنها (0) عند ظهور أي حرف جديد لأول مرة، يتفرع من عقدة NEW فرعان: الفرع الأيسر يصبح عقدة NEW الجديدة بوزن (0)، والفرع الأيمن يأخذ الحرف الجديد بوزن (1) كما يتم تمثيل الاتجاه الأيسر دائماً بالرقم 0 والاتجاه الأيمن بالرقم 1

الجزء الأول: قراءة الأحرف الأولى "A" و "AA" و "AAD" و "AADC"

1. عند قراءة الحرف الأول "A":

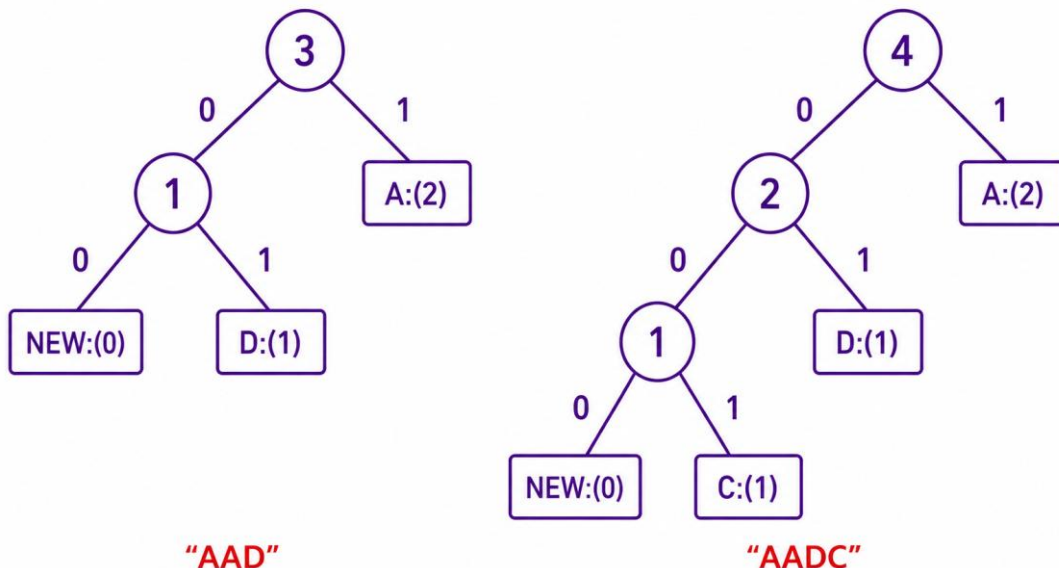
- يتم إدخال الحرف A لأول مرة، فينقسم الجذر إلى عقدة NEW:(0) على اليسار، والحرف A:(1) على اليمين.

- يصبح وزن الجذر في الأعلى (1).

2. عند قراءة الحرف الثاني "AA":

- الحرف A موجود مسبقاً في الشجرة، لذا لا ننشئ عقدة جديدة، بل نكتفي بزيادة تكراره ليصبح A:(2).

- يتم تحديث الأوزان صعوداً ليصبح وزن الجذر (2) الشجرة ما زالت متوازنة ومُرتبة بشكل صحيح.

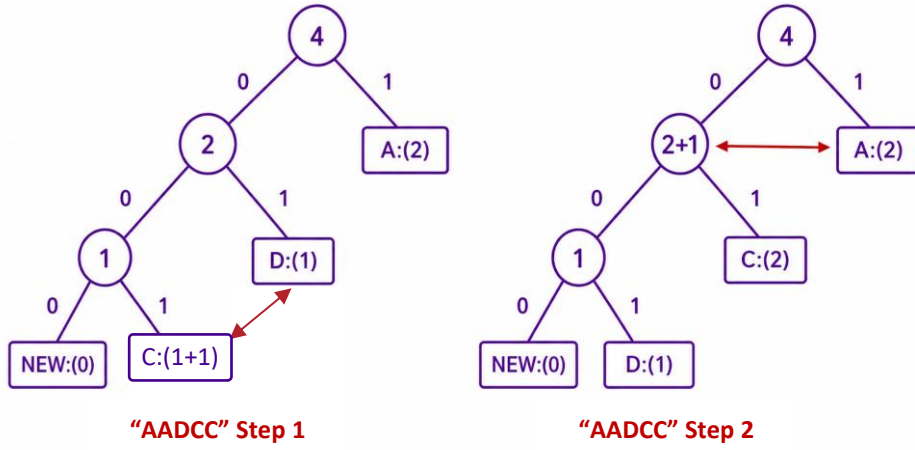


3. عند قراءة الحرف الثالث "AAD":

- الحرف D حرف جديد لم نره من قبل.
- يذهب النظام إلى عقدة NEW ويقسمها إلى فرعين: عقدة NEW:(0) جديدة على اليسار، والعقدة D:(1) على اليمين.
- يتم تحديث العقد العليا صعوداً: العقدة الفوقية مباشرة تصبح (1)، والجذر الرئيسي يصبح $1 + 2 = 3$.

4. عند قراءة الحرف الرابع "AADCC":

- الحرف C أيضاً حرف جديد.
- نذهب إلى عقدة NEW الحالية (في أقصى اليسار) ونقسمها مجدداً NEW:(0) على اليسار، والحرف الجديد C:(1) على اليمين.
- يتم تحديث أوزان المسار صعوداً: العقدة الفوقية تصبح (1)، التي تليها (2)، والجذر يصبح (4).



الجزء الثاني: قراءة الحرف الخامس "AADCC" وحدثت عمليات التبديل (Swaps)

هنا تظهر القوة الفعلية لخوارزمية Adaptive Huffman عند قراءة الحرف C للمرة الثانية، يرتفع وزنه، مما يسبب خللاً في قانون الترتيب التصاعدي للشجرة، مما يجبرنا على إجراء عمليات تبديل (Swap) لإعادة التوازن.

1. الخطوة 1 (Step 1): AADCC

- يظهر الحرف C للمرة الثانية، فيتغير وزنه من (1) إلى (2) $(1+1) = 2$.
- المشكلة: الحرف C الآن أصبح وزنه (2)، بينما جاره على اليمين في نفس المستوى هو الحرف D بوزن (1) في هافمان، لا يجوز لعقدة يسارية أو سفلية أن تكون أكبر وزناً من عقدة يمينية أو علوية.
- الحل: نقوم بعمل تبديل (Swap) بين عقدي C و D بأكملهما (موضح بالسهم الأفقي السفلي).

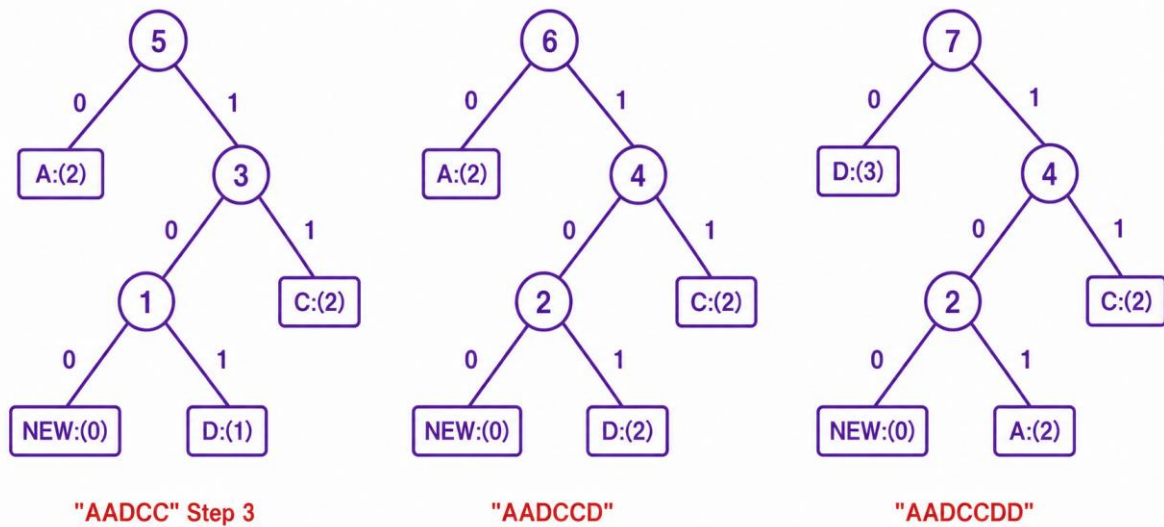
2. الخطوة 2 (Step 2): AADCC

- بعد تبديل الأماكن، أصبح D:(1) على اليسار، و C:(2) على اليمين.
- نقوم الآن بتحديث أوزان العقد صعوداً: العقدة الأبوية لـ D و C ترتفع لتصبح $1 + 2 = 3$ (مكتوبة في الصورة $2+1$)

- المشكلة الثانية: هذه العقدة الداخلية أصبح وزنها (3)، بينما جارتها اليمينية العليا وهي العقدة A وزنها (2) فقط. هذا يخرق الترتيب مجدداً.
- الحل: نقوم بعمل تبديل (Swap) ثانياً بين العقدة الداخلية بالكامل التي تحوي (C و D) وبين عقدة A (موضح بالسهم العلوي).

3. الخطوة 3 (Step 3): AADCC

- بعد التبديل الثاني، انتقلت العقدة A:(2) إلى أقصى اليسار، وانتقلت الشجرة الفرعية الحاملة لـ C و D إلى جهة اليمين
- الآن الشجرة أصبحت متزنة تماماً وموزعة تصاعدياً من اليسار لليمين A:(2) ثم العقدة الداخلية بوزن (3)، والجذر الكلي أصبح (5) $2 + 3 = 5$.



الجزء الثالث: قراءة الأحرف الأخيرة "AADCCD" و "AADCCDD"

1. عند قراءة الحرف السادس "AADCCD"

- الحرف المقروء الآن هو D نذهب إلى مكانه الحالي في الشجرة ونزيد وزنه ليصبح D:(2) بدلاً من (1)
- يتم تحديث العقد صعوداً: العقدة الأبوية لـ D تصبح $2 + 0 = 2$ ، والعقدة الأعلى تصبح $2 + 2 = 4$ ، ويصل الجذر في الأعلى إلى الوزن (6) الشجرة متوازنة ولا تحتاج لتبديل.

2. عند قراءة الحرف السابع والأخير "AADCCDD"

- يظهر الحرف D مجدداً للمرة الثالثة، فيرتفع وزنه ليصبح D:(3).
- المشكلة: عقدة D أصبح وزنها (3) بينما تقع في فرع يساري أسفل الحرف A الذي وزنه (2) في الجهة المقابلة.
- الحل: يتم عمل تبديل (Swap) بين مكان D ومكان A.
- ينتهي الأمر بـ D:(3) في أقصى اليسار، و A:(2) في الداخل، ويُعاد حساب الأوزان صعوداً ليصبح وزن العقدة الداخلية (4) والجذر النهائي للشجرة يساوي (7).

■ ويصبح ترميز ضغط السلسلة الذي يصل إلى الـ Decoder:

Symbol	NEW	A	A	NEW	D	NEW	C	C	D	D
Code	0	00001	1	0	00100	00	00011	001	101	101

فيقوم الـ Decoder بقراءة هذه السلسلة ويقوم بتشكيل الشجرة بطريقة معاكسة للـ Encoder.

■ خصائصها:

1. الميزة الأساسية في هافمان التكميلي هي أن الشجرة ديناميكية، أي أنها تتحدث وتتغير مع كل حرف يتم قراءته، على عكس هافمان التقليدية الثابتة.
2. في هذه الخوارزمية لم تعد الاحتمالات تعتمد على معرفة مسبقة بل أصبحت تعتمد على البيانات الفعلية المستلمة حتى الآن.
3. لا يلزم مسح البيانات مسبقاً قبل بدء عملية الترميز فوراً أي كلما أتاننا حرف نضغطه ونحدث الـ Tree model

■ مساوئ Adaptive Huffman Coding:

1. لا نقوم دائماً بإسناد أصغر قيمة ممكنة من عدد البتات للـ Symbols وذلك بسبب أنها من الممكن أن تعطي قيم عشرية (bits with fractions) .

■ مثال:

الرمز باحتمال 0.9 يجب أن يكون على 0.15 bits ولكن الخوارزمية تسند له 1 bit (أكبر من القيمة اللازمة بـ 6 أضعاف).

خوارزمية الترميز الحسابي Arithmetic Coding

- تعتبر هذه الخوارزمية من الأساليب الأكثر فاعلية في ترميز البيانات حيث يتم ترميز الرسالة بالكامل برقم عشري يتراوح بين 0 و 1، ويتم إنجاز هذه الخوارزمية بطريقة تضمن لأي سلسلة بيانات أن يتم تمثيلها بشكل فريد من خلال هذا الرقم، ومن ثم يمكن استعادتها في عملية الفك ويسمى هذا الرقم **بالرقم السحري**.
- **الفكرة الأساسية للترميز الحسابي:** بدلاً من ترميز كل حرف بشكل مستقل نأخذ بعين الاعتبار ترتيبه وتكراره في البيانات.
- في البداية يتم تعيين لكل رمز إلى فترة فرعية يكون حجمها مساوياً لاحتمالات الرموز.
- الترميز الحسابي يستبدل تياراً من الرموز برقم واحد ذي فاصلة عائمة في النطاق

ملاحظة: هذه الخوارزمية فعالة لضغط النصوص حصراً.

خصائصها:

- ميزتها الأساسية هي القدرة على التعامل مع الاعتمادات بين الرموز التي تشير إلى العلاقة أو التبعية المحتملة في الظهور بين رموز مختلفة ضمن بيانات أو نص معين أي توقع وجود رمز ما قد يزداد أو يقل باعتبار ظهور رموز أخرى سابقة له في البيانات وهو شيء لا تستطيع الخوارزميات التقليدية القيام به كما أنها تقرب إلى حد بعيد من أقصى حدود ضغط لنموذج مصدر معين وهو ما يسمى بحد entropy المصدر.

خطوات تطبيق الخوارزمية: لدينا مرحلتين:

- المرحلة الأولى: مسح النص المراد ضغطه أي نمر على جميع محارفه ومن ثم ننشئ جدول التوزيع الاحتمالي لمحارف النص وذلك من خلال الخطوات التالية:
- أ - **نحسب عدد المحارف (بدون تكرار):** أي إحصاء عدد أنواع المحارف وننشئ جدول عدد أسطره يساوي عدد أنواع المحارف.
- ب - **نحسب عدد تكرار كل حرف:** أي Frequency ونحسب منه احتمال وروده:

$$\text{احتمال ورود محرف في نص معين} = \frac{\text{عدد مرات ورود هذا المحرف في النص}}{\text{عدد محارف النص}}$$

- ج - **تجزئة الفضاء الاحتمالي:** وكما نعلم القيم الاحتمالية تتراوح بين [0, 1] فيتم تجزئة المجال إلى مجالات جزئية (Sub-intervals) وذلك من أجل كل محرف ويتم تحديد طول المجال الجزئي للمحرف وفقاً لاحتمال ورود المحرف في النص وبعد كل اختيار لمجال جزئي لأحد المحارف يتم اعتبار ما تبقى هو المجال الجديد الذي سيتم تجزئته.

مثال: ليكن لدينا النص التالي "SWISS MISS":

نطبق الخطوات السابقة:

$$\text{احتمال ورود محرف في نص معين} = \frac{\text{عدد مرات ورود هذا المحرف في النص}}{\text{عدد محارف النص}}$$

- عدد المحارف بدون تكرار هو 5 محارف {S, W, I, M, Space} لذلك سنقوم بإنشاء جدول مكون من 5 أسطر
- نبدأ محرف محرف وصولاً لآخر النص وذلك من أجل حساب احتمالات الورد:

من أجل أول محرف S :

عدد مرات وروده في النص هو 5 فيكون احتمال وروده:

$$S = \frac{5}{10} = 0.5$$

المحرف W:

عدد مرات وروده في النص مرة واحدة فيكون احتمال وروده:

$$W = \frac{1}{10} = 0.1$$

المحرف I:

عدد مرات وروده في النص مرتان فيكون احتمال وروده:

$$I = \frac{2}{10} = 0.2$$

المحرف Space :

عدد مرات وروده في النص مرة واحدة فيكون احتمال وروده:

$$\text{Space} = \frac{1}{10} = 0.1$$

ملاحظة: سوف يكون لدينا لكل حرف range وما يهمننا هو طول ال range ولا يهمننا ترتيب الأحرف .

3. نقوم بتجزئة المجال [0,1] إلى مجالات جزئية من أجل كل محرف:

من أجل أول محرف S:

بما أن احتمال وروده هو 0.5 إذاً طول المجال المراد اختياره سيكون 0.5 ، وباعتبار أنه تم تجزئة المجال [0,1] إلى مجالين [0.5, 1] و [0, 0.5] واختيار المجال [0.5, 1] ليكون القسم الأول الذي يحجزه المحرف S من الفضاء الاحتمالي [0, 1]:

$$\begin{matrix} (0, 0.5) & (0.5, 1) \\ P(S) = 0.5 \rightarrow \text{Range } [0.5, 1] \end{matrix}$$

من أجل المحرف التالي W:

بما أن احتمال وروده هو 0.1 ، إذاً يتم اختيار مجال طوله 0.1 فيتم تجزئة المجال [0,0.5] إلى مجالين [0.4, 0.5] و [0, 0.4] واختيار المجال [0.4, 0.5] ليكون القسم الذي يحجزه المحرف W من الفضاء الاحتمالي ونتابع من أجل بقية المحارف فنحصل على التقسيم التالي للمجال [0, 0.5]

$$\begin{matrix} (0, 0.4) & (0.4, 0.5) \\ P(W) = 0.1 \rightarrow \text{Range } [0.4, 0.5] \end{matrix}$$

من أجل المحرف التالي A:

بما أن احتمال وروده هو 0.2 ، إذاً يتم اختيار مجال طوله 0.2 فيتم تجزئة المجال [0,0.4] إلى مجالين [0.2, 0.4] و [0, 0.2] واختيار المجال [0.2, 0.4] ليكون القسم الذي يحجزه المحرف من الفضاء الاحتمالي ونتابع من أجل بقية المحارف فنحصل على التقسيم التالي للمجال [0,0.4]

$$\begin{matrix} (0, 0.2) & (0.2, 0.4) \\ P(I) = 0.2 \rightarrow \text{Range } [0.2, 0.4] \end{matrix}$$

من أجل المحرف التالي M:

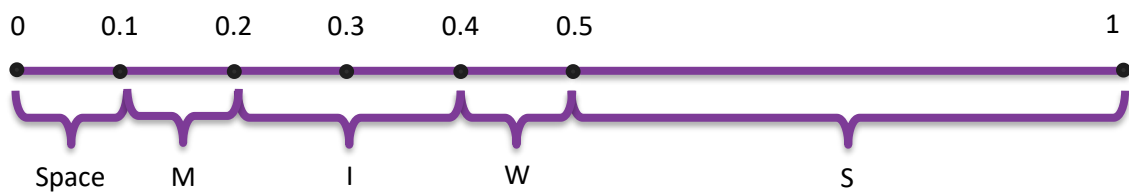
بما أن احتمال وروده هو 0.1 ، إذاً يتم اختيار مجال طوله 0.1 فيتم تجزئة المجال [0, 0.2] إلى مجالين [0.1, 0.2] و [0, 0.1] واختيار المجال [0.1, 0.2] ليكون القسم الذي يحجزه المحرف M من الفضاء الاحتمالي ونتابع من أجل بقية المحارف فنحصل على التقسيم التالي للمجال [0,0.2]:

$$P(M) = 0.1 \rightarrow \text{Range } [0.1, 0.2]$$

■ من أجل المحرف التالي Space:

بما أن احتمال وروده هو 0.1، إذاً يتم اختيار مجال وحيد [0,0.1]

$$P(\text{Space}) = 0.1 \rightarrow [0, 0.1]$$



ويكون جدول الفضاء الاحتمالي

الرمز	عدد مرات تكراره	احتماله	المجال الجزئي له
Space	1	0.1	[0.0,0.1[
M	1	0.1	[0.1,0.2[
I	2	0.2	[0.2,0.4[
W	1	0.1	[0.4,0.5[
S	5	0.5	[0.5,1.0[

ملاحظة: يتم إنشاء جدول توزيع احتمالي خاص بكل نص، وهو جدول غير وحيد إذ يمكن تغيير ترتيب المحارف فيه، كما يمكن تغيير قيم حدود المجالات باختيار مجالات مختلفة (يمكن أن نبدأ أولاً باختيار [0.2,0.7] من أجل المحرف s) لكن قيم الاحتمالات تبقى ذاتها.

■ المرحلة الثانية:

يتم مسح محارف النص من جديد وبالاعتماد على جدول التوزيع الاحتمالي الناتج عن الخطوة الأولى يتم تطبيق خوارزمية الترميز التالية:

```
low = 0.0;
high = 1;
while ( ( c = getc( input ) ) != EOF ) {
    range = high - low;
    high = low + range * high_range( c );
    low = low + range * low_range( c );
}
output ( low );
```

توضيح خطوات الخوارزمية:

1. تبدأ الخوارزمية بتعريف متحولين وإسناد قيمة ابتدائية لكل منهما $low = 0$, $high = 1$
2. من أجل كل محرف من محارف النص يتم الدخول بحلقه `while` وحساب ما يلي:
 - يتم إيجاد طول المجال المراد تجزئته:

$$range = high - low$$

- يتم تعديل قيمة $high$ من أجل الاستفادة منها في التكرار التالي وذلك بتطبيق المعادلة:

$$high = low + range * high_range(c)$$

- حيث أن $high_range(c)$ يمثل الحد الأعلى للمجال الذي يقوم بحجزه المحرف C (أي سيتم العودة عن طريق هذا التابع إلى جدول توزيع الاحتمالي لإحضار هذه القيمة).
- يتم تعديل قيمة low من أجل الاستفادة منها في التكرار التالي وذلك بتطبيق المعادلة:

$$low = low + range * low_range(c)$$

- حيث أن $low_range(c)$ يمثل الحد الأدنى للمجال الذي يقوم بحجزه المحرف C
- وبالوصول لنهاية النص تنتهي حلقة `while` ويتم تنظيم جدول عدد أسطره يساوي عدد تكرارات الحلقة ليتم وضع القيم السابقة ضمنه
 - 3. يكون خرج الخوارزمية هو قيمة low لآخر تكرار للحلقة : (أي لآخر سطر في الجدول، وتدعى هذه القيمة بالرقم السحري).

نكمل في المثال السابق:

- من أجل المحرف الأول S :
 - حدود المجال لهذا المحرف:
- $$high_range(S) = 1$$
- $$low_range(S) = 0.5$$
- طول المجال:
- $$range = 1 - 0 = 1$$
- (لأنه التكرار الأول فتكون القيم $low = 0$, $high = 1$)
- تعديل قيمة كل من $high$ و low :
- $$high = low + range * high_range(S) = 0 + 1 * 1 = 1$$
- $$low = low + range * low_range(S) = 0 + 1 * 0.5 = 0.5$$
- من أجل المحرف التالي W :
 - يتم الدخول بالتكرار التالي للحلقة :
 - حدود المجال لهذا المحرف:
- $$high_range(W) = 0.5$$

$$\text{low_range}(W) = 0.4$$

▪ طول المجال:

$$\text{range} = 1 - 0.5 = 0.5$$

(لأنه يتم الاعتماد على قيم low , high من التكرار السابق)

▪ تعديل قيمة كل من high , low:

$$\text{high} = \text{low} + \text{range} * \text{high_range}(W)$$

$$= 0.5 + 0.5 * 0.5 = 0.75$$

$$\text{low} = \text{low} + \text{range} * \text{low_range}(W)$$

$$= 0.5 + 0.5 * 0.4 = 0.7$$

ويتم المتابعة بنفس الطريقة للوصول إلى نهاية محارف النص وبالتالي يتكون الجدول التالي:

الرمز	Low_range(c)	High_range(c)	range	l	h
				0.0	1
S	0.5	1	1	0.5	1
W	0.4	0.5	0.5	0.7	0.75
I	0.2	0.4	0.05	0.71	0.72
S	0.5	1	0.01	0.715	0.72
S	0.5	1	0.005	0.7175	0.72
	0	0.1	0.0025	0.7175	0.71775
M	0.1	0.2	0.00025	0.717525	0.71755
I	0.2	0.4	2.5E-05	0.71753	0.717535
S	0.5	1	5E-06	0.7175325	0.717535
S	0.5	1	2.5E-06	0.717534	0.717535

نجد أن قيمة الرقم السحري في السطر الأخير من الجدول التي تمثل قيمة المتحول low هي:

$$\text{low} = 0.717534$$

وبذلك يتم ضغط النص من خلال تخزين جدول توزيع الاحتمالي بالإضافة لقيمة الرقم السحري بدلاً من تخزين النص.

ليس التوفيق أن تبلغ ما تريد، بل أن يبلغك الله ما فيه رضاه وخيرك

عملية فك الضغط المطبق باستخدام خوارزمية The decoder Coding Arithmetic Algorithm

يتم بالطريقة المعاكسة وذلك بالاعتماد على الرقم السحري وجدول التوزيع الاحتمالي وبتطبيق خوارزمية فك الترميز التالية:

```
number = input_code();
for ( ; ; ) {
    symbol = find_symbol_in_range( number );
    putc( symbol );
    range = high_range(symbol) - low_range(symbol);
    number = (number - low_range(symbol))/ range;
}
```

الخطوات لهذه الخوارزمية:

1. تداهه الخوارزمية بتعريف متحول number والذي يتم تهيئته بالرقم السحري المخزن في عملية ضغط النص
2. يتم الدخول في حلقة for وفي كل تكرار يتم إيجاد ما يلي:
 - نوجد المحرف المقابل للرقم number : أي المحرف الذي يحوي مجاله على قيمة number وذلك بالعودة لجدول التوزيع الاحتمالي من خلال التابع find_symbol_in_range(number).
 - نقوم بحساب طول المجال للمحرف الذي تم إيجاده: أي حساب قيمة range.
 - نقوم بتعديل قيمة المتحول number : أي حساب القيمة الجديدة له لاستخدامها في التكرار وذلك وفقاً للمعادلة:

$$number = \frac{number - low_range(symbol)}{range}$$

- تتوقف الحلقة عندما يصبح number = 0 وتكون المحارف الناتجة هي محارف النص قبل الترميز.

بالعودة إلى المثال السابق وتطبيق خوارزمية فك الترميز:

1. Number = 0.71753375
2. نبحث عن المحرف الذي ينتمي لمجاله الرقم السابق من جدول التوزيع الاحتمالي فنجد أنه المحرف S لأن $0.7 \in [0.5, 1]$ وطول مجاله range = 0.5 ، فنقوم بتسجيل المحرف S ونوجد الرقم الجديد:

$$number = \frac{0.71753375 - 0.5}{0.5} = 0.4350675$$

3. نكرر العملية السابقة ونبحث عن المحرف الذي يحوي مجاله على قيمة number فنجد أنه المحرف W لأن $0.4 \in [0.4, 0.5]$ فنقوم بتسجيل المحرف W ونوجد الرقم الجديد كما سبق. ونكمل حتى يصبح number = 0 ونحصل على الجدول التالي:

	char	range	number
0.71753375	S	0.5	0.4350675
0.4350675	W	0.1	0.350675
0.350675	I	0.2	0.753375
0.753375	S	0.5	0.50675
0.50675	S	0.5	0.0135
0.0135	Space	0.1	0.135
0.135	M	0.1	0.35
0.35	I	0.2	0.75
0.75	S	0.5	0.5
0.5	S	0.5	0

الآن سننتقل إلى نمط خوارزميات الضغط التي تعتمد على القواميس (Dictionary Based Compression)

- هي فئة من الخوارزميات التي تستخدم "قاموس" من السلاسل النصية التي ظهرت سابقاً في البيانات لضغط البيانات الجديدة.
- هذه الطريقة تعتمد على فكرة أنه يمكن استبدال السلاسل المتكررة داخل النص بمؤشرات إلى تلك السلاسل لتوفير مساحة التخزين.

وهذه الخوارزميات نوعين:

4. Static Dictionary:

نقوم فيها ببناء القاموس قبل عملية الضغط ولا نغير فيه شيء، لا تكون عامة الاستخدام ويتم تعديلها لتناسب البيانات التي يتم ضغطها.

5. Dictionary Adaptive (LZW , LZ77):

تبدأ عملية الضغط دون وجود قاموس أو بوجود قاموس افتراضي ، ونقوم بإضافة كيانات جديدة إلى القاموس أثناء عملية الضغط

خوارزمية LZ77

- التقنيات الإحصائية ترمز رمزاً واحداً في كل مرة باستخدام عدد متغير من البتات أما تقنيات المعاجم ترمز سلاسل متغيرة الطول من الرموز كرمز مميز واحد لقاموس العبارات
- تستخدم هذه الخوارزمية sliding window , look_ahead buffer لمعرفة الأحرف القادمة
- ليس من الضروري أن يكون الـ symbol هو محرف من نص، من الممكن أن يكون بأي كمية من البيانات، ولكنه عادة يكون بحجم 1 byte.
- تقوم هذه الخوارزمية بمقارنة سلسلة المحارف الموجودة في sliding window مع سلسلة المحارف الموجودة في الـ look_ahead buffer وتحاول إيجاد تطابق بين أي سلسلة جزئية من الـ sliding window و الـ Look_ahead Buffer وهنا يكون لدينا حالتين:

1. لا يوجد تطابق:

فتقوم بترميز أول رمز في الـ look_ahead ككيان لوحده وهنا لا يوجد ضغط

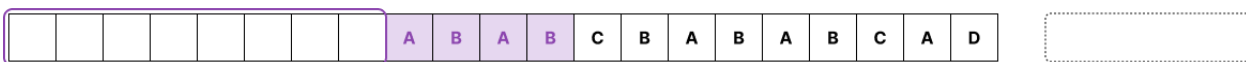
2. يوجد تطابق:

فتقوم بترميز ثلاثة معلومات:

- الـ index التي بدأ عندها التطابق في الـ sliding window.
- عدد الرموز المتطابقة.
- أول رمز بعد التطابق في الـ look_ahead buffer.

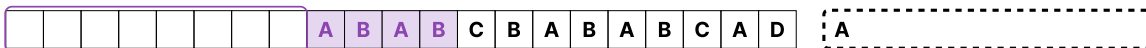
■ مثال: ليكن لدينا سلسلة المحارف ABABCBABABCAD و نريد ضغطها LZ77 :

1 Initially



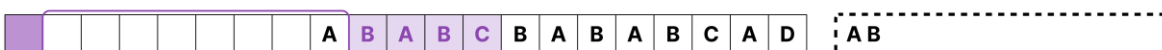
- تكون الـ sliding window بأول عملية الضغط فارغة (وهي الخانات ذات الإطار) ولا يوجد تطابق مع الـ look_ahead buffer (الخانات ذات اللون النهدي) ولذلك نضع أول رمز هو A ككيان token ونضمها في الملف المرمز فيصبح A ونزيع الـ sliding window :

2 After not finding any phrase from ABAB in the sliding window and encoding A as the symbol token A



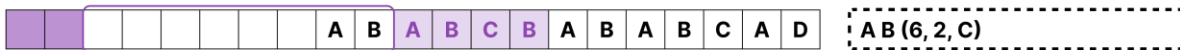
- في هذه المرحلة أيضاً نلاحظ أنه لا يوجد تطابق بين الـ sliding window و الـ look_ahead buffer لذلك نضيف الـ B ككيان token ولوحدها ونضعها بالملف المرمز فيصبح AB ثم نزيع الـ sliding window :

3 After not finding any phrase from BABC in the sliding window and encoding B as the symbol token B



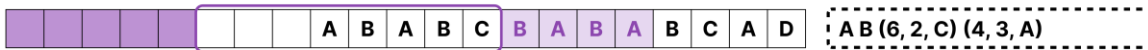
- في هذه المرحلة نلاحظ وجود تطابق والذي هو "AB" بين sliding window و look_ahead buffer فتقوم بترميزها بثلاث معلومات والتي هي (6,2,C) حيث :
 - C : هو المحرف الذي يأتي بعد التطابق في look_ahead buffer.
 - 2 : هو طول أحرف التطابق "AB".
 - 6 : هو index في ال sliding window الذي بدأ عنده التطابق (بما أن التطابق بدأ عند "A")
- فال index هو 6 فنضعها ككيان token في الملف المرمز فيصبح AB (6,2,C) ثم نزيح ال sliding window إلى محرف ما بعد التطابق:

4 After Finding AB at offset 6 in the sliding window and encoding AB as the phrase token (6, 2, C)



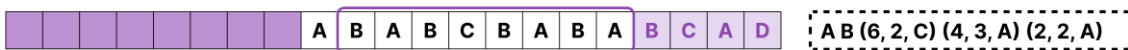
- في هذه المرحلة نلاحظ أيضاً وجود تطابق والذي هو "BAB" بين ال sliding window و ال look_ahead buffer فنقوم بترميزها أيضاً (4,3,A) فنضعها ككيان token في الملف المرمز فيصبح AB(6,2,C)(4,3,A) ثم نزيح ال sliding window إلى محرف ما بعد التطابق :

5 After finding BAB at offset 4 in the sliding window and encoding BAB as the phrase token (4, 3, A)



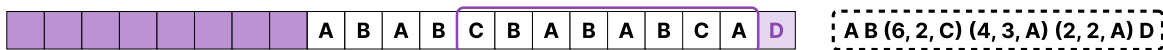
- يوجد أيضاً تطابق "BC" فنقوم بترميزها (2,2,A) ونضعها في الملف المرمز فيصبح AB(6,2,C)(4,3,A)(2,2,A) ثم نزيح ال sliding window إلى محرف ما بعد التطابق:

6 After finding BC at offset 2 in the sliding window and encoding BC as the phrase token for (2, 2, A)

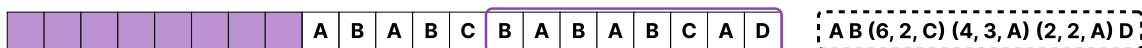


- لا يوجد تطابق فنضيف ال "D" ككيان token إلى الملف المرمز فيصبح AB (6,2,C)(4,3,A)(2,2,A)D وهكذا تكون انتهت عملية الترميز:

7 After not finding D in the sliding window and encoding D as the symbol token D

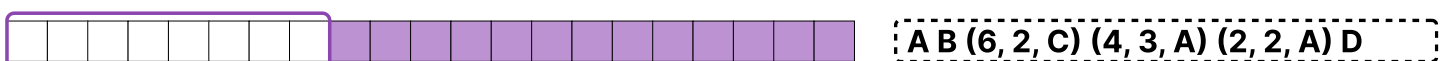


8 No data remaining in the look-ahead buffer



فك الضغط:

1 Initially



- والآن نريد فك الضغط عن الملف المرمز AB(6,2,C)(4,3,A)(2,2,A)D

▪ نضيف A فيصبح الملف القرمز (2,2,A)(4,3,A)(6,2,C):

2 After Decoding the symbol token A



B (6, 2, C) (4, 3, A) (2, 2, A) D

▪ نضيف B فيصبح الملف القرمز (2,2,A)(4,3,A)(6,2,C)D:

3 After Decoding the symbol token B

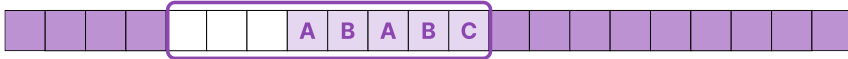


(6, 2, C) (4, 3, A) (2, 2, A) D

▪ لفك الضغط عن الـ (6,2,C) نقوم أولاً بأخذ الـ index الذي رقمه 6 من الـ sliding window والتي هي عند الـ "A"

ونمشي بمقدار خطوتين فنأخذ "AB" ونضع بعدها C فيصبح الملف القرمز (2,2,A)(4,3,A)D

4 After Decoding the phrase token (6, 2, C)



(4, 3, A) (2, 2, A) D

▪ فنقوم بنفس العملية لفك الضغط عن (4,3,A) فيصبح الملف القرمز (2,2,A)D:

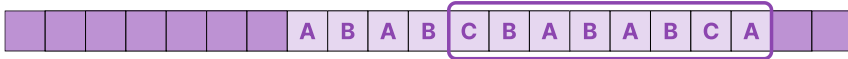
5 After Decoding the phrase token (4, 3, A)



(2, 2, A) D

▪ ونقوم أيضاً بنفس العملية لفك الضغط عن (2,2,A) فيصبح الملف القرمز D:

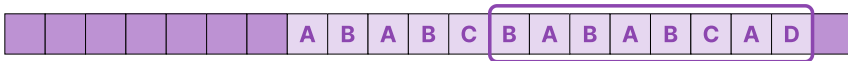
6 After Decoding the phrase token (2, 2, A)



D

▪ ثم أخيراً نقوم بوضع D:

7 After Decoding the symbol token D



وهكذا تكون انتهت عملية فك الضغط.

وفي هذه الخوارزمية يعتمد Compression ratio على ثلاثة أشياء:

1. حجم الـ sliding window.
2. حجم الـ look_ahead buffer.
3. الـ entropy الخاصة بالداتا.

ملاحظات:

- الحجم المثالي للـ sliding window هو 4 KB.
- يكون حجم الـ look_ahead buffer غالباً أقل من 100 B.
- LZ77 هي بشكل عام أفضل من خوارزمية Huffman لكنها أبطأ.
- عملية فك الضغط في LZ77 تعتبر سريعة

خوارزمية LZW

أنه عند البدء فيها يكون لدينا قاموس الـ (ASCII) مثلاً ونقوم بإضافة عناصر لهذا القاموس. فلنفرض مثلاً أننا بدأنا بقاموس الـ (ASCII) و الذي هو يرمز من 0 إلى 255 فنقوم بإضافة عناصر جديدة لهذا القاموس بداية من الـ 256 و تكون هذه العناصر الجديدة عبارة عن سلسلة من المحارف مثل السلسلة "AB" تغيرها كرمز واحد و نضعه في القاموس بالترميز 256 مثلاً.

أي طالما وجدت المحرف في القاموس نستمر، فيما عدا ذلك نضيف المحرف الجديد مع الذي يسبقه كـ token جديد إلى القاموس.

■ مثال: السلسلة "WED WE WEE WEB WET":

فנקون خطوات ضغطها بخوارزمية LZW:

1. نقرأ أول حرف هو space " " ونقوم بترميزه بالـ (ASCII)
2. ثم نقرأ الحرف الثاني والذي هو "W" ونقوم أيضاً بترميزه بالـ (ASCII).
3. فيصبح لدينا الثنائية "W" فنقوم بإضافتها إلى القاموس و نعطيها الترميز 256:

$$\text{"W"} = 256$$

4. ثم نقوم بقراءة الـ "E" و بما أن الثنائية "WE" غير موجودة في القاموس نقوم بإضافتها إلى القاموس

$$\text{"WE"} = 257 \text{ و نقوم بترميز الـ "E" بالـ (ASCII)}$$

و بالمتابعة بنفس الطريقة على كامل السلسلة ينتج لدينا القاموس التالي:

- | | | |
|---------------|---------------|---------------|
| ■ "W" = 256 | ■ "WE" = 257 | ■ "ED" = 258 |
| ■ "D" = 259 | ■ "WE" = 260 | ■ "E" = 261 |
| ■ "WEE" = 262 | ■ "EW" = 263 | ■ "WEB" = 264 |
| ■ "B" = 265 | ■ "WET" = 266 | ■ |

Characters Input X	Code Output	New code value and associated string
"W"	' '	"W " = 256
"E"	'W'	"WE" = 257
"D"	'E'	"ED" = 258
" "	"D"	" D" = 259
"WE"	256	"WE " = 260
" "	'E'	" E" = 261
"WEE"	260	"WEE" = 262
" " W	261	"E W" = 263
"EB"	257	"WEB" = 264
" "	B	" B" = 265
"WET"	260	"WET " = 266

حيث أن ال input هو المحرف الحالي، وال output هو ما سبقه إذا كان في القاموس. نشكل token جديدة من input + output ونضعه في القاموس بشرط input + output غير موجودين في القاموس ، وفي حال وجدوا نعود خطوة للسابق، ويكون الملف المضغوط كما يلي:

" W E D <256> E <260> <261> <257> B <260> T "

لا نمرر القاموس لل decoder في هذه الخوارزمية وإنما يبدأ decoder بقاموس يحتوي على الرموز من 0 إلى 255 ثم يقوم بقراءة الدخل ويعتمد عليه للحصول على المحارف اللاحقة و يقوم ببناء القاموس ذاته الذي تمّ بناؤه في ال encoder:

NEW CODE	Output	New Table Entry
' '	" "	
'W'	"W"	"W " = 256
'E'	"E"	"WE" = 257
'D'	"D"	"ED" = 258
256	"W "	" D" = 259
'E'	"E"	"WE " = 260
260	"WE "	" E" = 261
261	" E"	"WEE" = 262

The End...

«قُلْ لِلَّذِي أَحْصَى السِّنِينَ مُفَاخِرًا
يَا صَاحِبَ لَيْسَ السِّرِّ فِي السَّنَوَاتِ
لِكِنَّهُ فِي الْمَرْءِ كَيْفَ يَعِيشُهَا
فِي يَقْظَةٍ أَوْ فِي عَمِيقِ سُبَاتٍ»

